Assignment 1

Tools & Techniques for Large Scale Data Analytics
University of Galway, Academic Year 2026-27

- ***Submission deadline (strict): <u>Wednesday, 7th October 2026, 23:59</u>. Late submission requires a medical or counsellor's certificate.***
- *Put all your source code files (and other answers, if any) into a **<u>single .zip archive</u>** with name "YourName_Assignment1.zip" and submit on Canvas by the deadline.*
- *Include all source code files (files with name ending .java) required to compile and run your code*
- *Create a new Java source code file for each question*
- *Unless specified otherwise in the question, use only plain Java 25 for this assignment (no external libraries or frameworks)*
- *All submissions will be checked for plagiarism. In case of plagiarism, both the copier(s) and the provider of the copy (if it is a student in this class) receive 0 marks on this entire assignment. Note that group work or the use of Artificial Intelligence are not allowed in this module**.***
- ***<u>Provide Java comments which explain your code</u>. Missing or insufficient comments lead to mark deductions.***

Question 1 [30 marks]

Suppose you want to analyze the temperature data produced by weather stations (meteorological stations).

Create a class `WeatherStation` with three attributes (fields): the *`city`* where the station is located, the station's *`measurements`* (a list of objects of class `Measurement`), and a *static* field *`stations`* (a list of all existing weather stations – you need this list only for the next question). Also, create a class `Measurement`. Objects of class `Measurement` should have attributes *`time`* (an integer, representing the time of the measurement) and *`temperature`* (a double number).

Add a method `maxTemperature(startTime, endTime)` to class `WeatherStation` which returns the maximum temperature measured by this weather station between `startTime` and `endTime`. Implement this method using Java 8 Stream operations, as far as possible. Also, add a `main`-method where you call your method using some test data (of your own choice) and print the result.

Question 2 [70 marks]

Add a method `countTemperatures(t1,t2,r)` to your class `WeatherStation` from the previous question. The method should return a list which contains two pairs of values: 1) temperature `t1` paired with the number of times a temperature in the interval `[t1-r..t1+r]` has been measured so far by any of the weather stations in *`stations`*, and 2) temperature `t2` paired with the number of times a temperature in the interval `[t2-r..t2+r]` has been measured so far by any of the weather stations in *`stations`*. Note that a single measurement contributes to both counts in case it is within both intervals.

It is sufficient for Question 2 to consider only the case that there are **exactly <u>two</u>** weather stations in list *stations* (a solution which works with an arbitrary number ≥ 2 of stations would, of course, also be fine

<u>PTO</u>

and is not necessarily more complex, but it would not be worth additional marks). Considering only a single station would not be sufficient.

Example: if there are two weather stations in list *stations* and the first station has measured temperatures 20.0, 11.7, -5.4, 18.7, 20.9 and the second station's measurements are 8.4, 19.2, 7.2, then `countTemperatures(19.0,10.8,2.1)` should return the list ((19.0,4), (10.8,1)) (because there are in total (i.e., considering all stations) 4 measurements in the range between 19.0-2.1 and 19.0+2.1 and 1 measurement in the range from 10.8-2.1 to 10.8+2.1).

For computing the result, you must use an "emulated" **MapReduce** approach, as far as possible. That is, your code should resemble the MapReduce approach but using only plain Java >=8 (i.e., without using any actual MapReduce framework such as Hadoop). Also, you need to make use of Java 8 Streams (as far as possible) and parallel stream processing (where appropriate).

Finally, add code to your `main`-method which calls your `countTemperatures` method using some test measurement data from two different stations, and prints the result.

Important: This question doesn't ask for the fastest or shortest algorithm! Instead, your solution should reflect, as far as possible, the MapReduce phases "Map", "Shuffle" and "Reduce" as described in the lecture, and it should appropriately use (key,value)-pairs in each phase (so you will also need some data structure for representing pairs, e.g., a class with two fields). There should be some kind of grouping of values (or (key,value)-pairs) by their keys, resembling the "Shuffle" step between "Map" and "Reduce" (see lecture notes). Additional operations, e.g., for filtering, can be added if needed.

Hints: Studying the classic "Word Count" example from the lecture will be helpful (although there are some differences) - e.g., weather stations might correspond to inputKeys, certain temperatures might correspond to words… Also, take a look at the available methods in Java API interface *Stream* (see https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/stream/package-summary.html), e.g., flatMap for "flattening" nested streams might be handy…